Faculty Teaching & Office-Hours Schedule Generator

COSC 631 – Software Engineering · Second Project and Midterm Report

Carlson Cox

October 2026

Contents

- 1. Overview
- 2. System design
- 3. Input data
- 4. Static libraries
- 5. The Makefile
- 6. Scheduling logic
- 7. Error handling and robustness
- 8. Input file location: local vs. Internet
- 9. Testing
- 10. Sample output
- 11. Midterm: answering questions with Python and Prolog
- 12. Limitations and future work
- Appendix A – Submission contents

1. Overview

This project is a C program, `schedgen`, that automatically generates each professor's **teaching schedule** and **office-hours schedule** for a faculty. It reads **exactly two input files**:

1. **Class schedule**: the department's CTEC Fall 2026 schedule (56 sections, 25 instructors), exactly as distributed as an Excel workbook. It holds section IDs, titles, credits, days, times, rooms, instructors and enrollment. The sections run in two 7-week sessions, **7R1** and **7R2**.
2. **Faculty information file**: one record per instructor with department, office, contact details, courses taught, office-hours requirements, and every spelling of the instructor's name that appears in the schedule.

The program validates both files against each other, detects scheduling conflicts, repairs obvious data-entry errors (with a warning), and assigns office hours that never fall during class time. It writes a formatted text report plus CSV data. Two Python scripts, called from the Makefile, export the department's `.xlsx` to CSV before the C step and convert the C output into **Word** and **Excel** documents in the style of the sample tables in the assignment. All scheduling logic lives in a **static library**, `libschedule.a`; `main.c` is a thin driver linked against it.

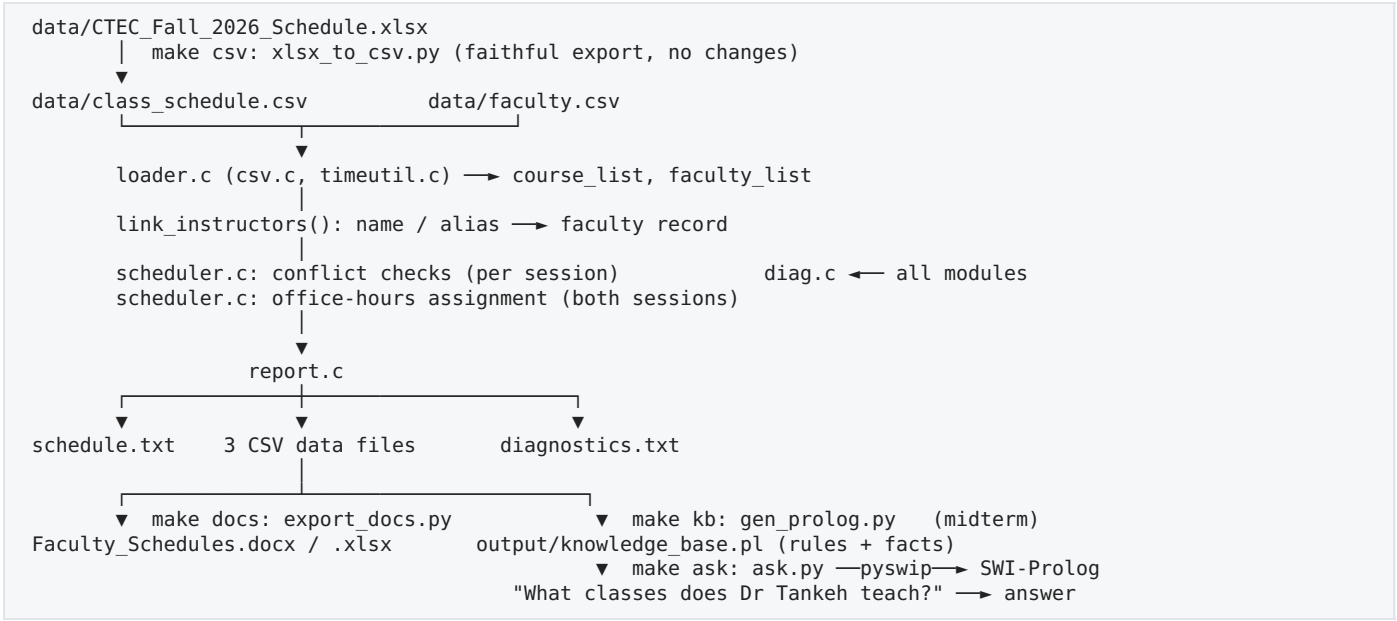
Midterm extension. The system now also answers questions such as *"What classes does Dr Tankeh teach?"* or *"Which office hours are on Wednesday?"*. A Python script reads the program's output and generates the Prolog **knowledge base**, one file holding the **rules** (how the facts relate) and the **facts** (the schedule); and a Python front end (using the `pyswip` bridge) turns an English question into a Prolog query, which SWI-Prolog answers by search. Section 11 describes this part.

1.1 Requirements traceability

Requirement (from the assignment)	Where it is satisfied
Implemented in C	`src/main.c`, `src/lib/*.c`, `include/*.h` (C11, ~3,700 lines)
Exactly two input files	`main.c` rejects any other count (exit status 1); inputs are the class schedule and `data/faculty.csv`
Syllabus / class-schedule file	The department's CTEC Fall 2026 schedule, read in its native column layout (§3)
Faculty information file	`data/faculty.csv`: lecturers, department, courses taught, office-hours policy (§3.2)
Generate teaching + office-hours schedules	`scheduler.c` (assignment), `report.c` (tables), §6
Use office-hours tables provided by the lecturer	`fixed_office_hours` column, validated and kept unless it collides with a class (§6.2)
Sample tables are the *style* of output, not hard-coded data	Nothing is hard-coded; output mirrors the sample's Course / Times / Location layout (§10)
Organize the project into modules	7 library modules + driver, each with its own header (§2)
Create and use a static library	`build/libschedule.a` via `ar rcs`, linked with `-Lbuild -lschedule` (§4)
Makefile: compile, create library, link, generate output	`Makefile` targets `all`, `lib`, `csv`, `run`, `docs`, `test` (§5)
Python from the Makefile for Word/Excel	`make csv` → `xlsx_to_csv.py`; `make docs` → `export_docs.py` (§5.5)
Explain Internet vs. local input files	§8
Submit source, headers, library, Makefile, inputs, Python, sample output, README	Appendix A
Midterm: answer questions such as "What classes does Dr Tankeh teach?"	`scripts/ask.py` + Prolog; `make ask Q="..."` (§11)
Midterm: Python reads the schedule data and generates facts and rules for Prolog	`scripts/gen_prolog.py` → `output/knowledge_base.pl` (§11.2–11.4)
Midterm: Prolog stores knowledge as facts and rules	`output/knowledge_base.pl`: part 1 rules, part 2 facts (§11.3–11.4)
Midterm: output in the style of the sample	"What is Dr Tankeh's schedule?" prints CLASS SCHEDULE / OFFICE HOURS / NOTE in the sample's layout (§11.7)
Midterm: Prolog acts as the interpreter that answers questions	`pyswip` consults both files and runs each query in SWI-Prolog (§11.5)

2. System design

2.1 Architecture



2.2 Modules

Module	Header	Responsibility
`common.c`	`common.h`	Size limits, status codes, bounded string copy/trim, case-insensitive compare, dynamic-array growth
`timeutil.c`	`timeutil.h`	Parse `4:00 PM`, `16:00`, `4:00:00 PM` (Excel), `MW`, `TR`, `TTh`; format “Monday & Wednesday 4:00 PM to 6:50 PM”
`csv.c`	`csv.h`	RFC 4180 reader/writer; header matching that ignores case, spaces and punctuation, with prefix match (`Instructor*`)
`diag.c`	`diag.h`	Collects INFO / WARNING / ERROR messages so modules never print directly
`loader.c`	`model.h`	Loads both files; splits `CTEC 350.170` into course and section; extracts the session; classifies meetings as scheduled / asynchronous / TBA; repairs AM/PM errors; resolves instructor names through aliases
`scheduler.c`	`scheduler.h`	Session-aware conflict detection (instructor, split rooms); office-hours assignment on a 15-minute weekly grid
`report.c`	`report.h`	Text report (tables, data notes, weekly grid), CSV data files, diagnostics file
`main.c`	–	Command-line parsing, orchestration, exit codes. **Not** in the library

`include/schedule.h` is an umbrella header: a client needs only `#include "schedule.h"` and `-lschedule`.

The unit-test program is a second client of the same library, showing the library is reusable without `main.c`.

2.3 Key design decisions

- **Read the department's format as-is.** Columns are found by header name, so `Instructor 2025`, `CR` and `End` (trailing spaces) need no hand editing, and column order does not matter.
- **Repair in C, not in Python.** The `.xlsx` → CSV export changes no values. Every correction happens in the C loader and is reported, so the same rules apply whether the CSV came from the script or from Excel's "Save As".
- **Report, don't refuse.** A conflict in the department's data is a finding, not a crash. All schedules are still produced, problems are listed first, and the exit status (3) lets scripts detect them.
- **The C program owns all formatting.** The CSV output carries display strings, so the Python exporter only lays them out, and the text, Word and Excel outputs can never disagree.

3. Input data

3.1 The class schedule (input 1)

Column	Example	Interpretation
`ID`	`CTEC 350.170`	Course `CTEC 350`, section `170`
`Course Title`	`Prin & Meth of Intru Det & Pre (7R1)`	Title; trailing `(7R1)` = session (first 7 weeks)
`CR`	`3`	Credits
`Days`	`MW`, `TR`, `MWF`	R = Thursday; blank = asynchronous or TBA
`Begin`, `End`	`4:00 PM`, `6:50 PM`	Excel time values, exported as displayed
`Room`	`108`, `201/112`, `Online`, `Dept`	Split room = both rooms; `Online`/`Dept` are not physical rooms
`Instructor 2025`	`Adedoyin, Anthony`	Matched to a faculty record by name or alias
`Enroll #`	`22`	Enrollment

3.2 The faculty file (input 2)

The schedule contains 28 distinct instructor strings for 25 people, in two name orders (`Adedoyin, Anthony` vs. `Ruth Agada`) and with variants (`Bemley`, `Bemley, J`, `Bemley, Jesse`; `Eugene Harris`, `Eugene T. Harris`). The faculty file gives each person a stable ID (`CT01`–`CT25`) and an `aliases` column listing every spelling. Matching ignores case, spacing and punctuation. An instructor string with no match is an ERROR, so a new or misspelled name can never silently become a separate professor.

The file also carries the office-hours policy per professor (required hours, block length, preferred days, availability window, lecturer-supplied hours). Office, email and phone are not in the department schedule and are left blank for the department to complete. The output then shows "Office TBA", or "Online (virtual)" for instructors whose sections are all online.

3.3 Findings in the CTEC Fall 2026 data

The program reported the following. Each was checked by hand against the spreadsheet.

Severity	Finding	Program action
ERROR	CTEC 711.555 meets MWF **5:00 PM to 5:00 PM** (zero length)	Listed as TBA; not placed on the grid
WARNING	CTEC 721.555 meets **7:00 PM to 9:50 AM**	End read as 9:50 PM (matches the 7:00–9:50 PM pattern of the program)
WARNING	CTEC 120.170 meets **12:00 PM to 3:15 AM**	End read as 3:15 PM
ERROR	**Room 112**, 7R1, MW: CTEC 120.171 (rooms 201/112, 5:00–7:00 PM) overlaps CTEC 294.170 (4:00–6:50 PM)	Reported; both schedules produced
ERROR	**Room 111**, 7R2, MW: CTEC 345.180 (5:00–7:50 PM) overlaps CTEC 450.180 (7:00–9:50 PM)	Reported
ERROR	**Ruth Agada**, 7R2, MW 7:00–9:50 PM: assigned to both CTEC 402.180 and CTEC 435.180	Reported

Not reported, correctly: the many instructors who teach the same time slot and room in both sessions (e.g. CTEC 350.170 in 7R1 and 350.180 in 7R2, both MW 4:00–6:50 PM in room 108). These are the same slot in different halves of the term, not a conflict. On this data a session-unaware check reports 27 double-bookings (11 instructor, 16 room); only 3 are real, so 24 would be false alarms.

4. Static libraries

4.1 What a static library is

A **static library** (Unix extension `.a`, “archive”) is a single file that bundles several compiled object files (`.o`) together with a **symbol index**. At link time the linker copies the machine code it needs from the archive directly into the executable. After linking, the program no longer depends on the library file at all.

This contrasts with a **shared (dynamic) library** (`.so` on Linux, `.dll` on Windows), which is loaded at *run time* and shared by every program that uses it.

	Static library (`.a`)	Shared library (`.so`)
When code is bound	Link time, copied into the executable	Run time, loaded by the dynamic loader
Executable size	Larger (contains library code)	Smaller
Deployment	Self-contained, nothing extra to install	Library must be present on the target machine
Updating the library	Relink every program	Replace the `.so`; programs pick it up
Symbol resolution	Linker pulls **only the object files**	Whole library mapped

Symbol resolution	Linker pulls **only the object files that are needed**	Whole library mapped

A static library is the right choice here: the program runs on a lab computer, must be easy to build and grade, and gains nothing from sharing code with other programs at run time.

4.2 How the library is created

```
        # 1. Compile each library source into an object file (-c = compile only, do not link)
        gcc -Iinclude -std=c11 -Wall -Wextra -c src/lib/timeutil.c -o build/obj/lib/timeutil.o
        #    ... same for common.c csv.c diag.c loader.c report.c scheduler.c

        # 2. Archive the objects into the static library
        ar rcs build/libschedule.a build/obj/lib/*.o
```

`ar` flags: **r** inserts or replaces members, **c** creates the archive without a warning, **s** writes the symbol index (equivalent to running `ranlib`), which lets the linker find which member defines a symbol.

`make inspect` shows the result:

```
== Members of build/libschedule.a (ar -t) ==
common.o  csv.o  diag.o  loader.o  report.o  scheduler.o  timeutil.o

== Exported functions (nm --defined-only, type T) ==
T load_courses       T link_instructors          T sched_check_conflicts
T load_faculty       T faculty_match             T sched_assign_office_hours
T report_write_text  T report_write_csv          T tu_parse_time        ... (59 functions)
```

4.3 How the library is linked

```
        gcc -Iinclude -c src/main.c -o build/obj/main.o          # compile the driver
        gcc build/obj/main.o -Lbuild -lschedule -o build/schedgen # link against the library
```

- `-Iinclude`: where the **compiler** finds the headers. Headers declare the functions; the library defines them.
- `-Lbuild`: adds `build/` to the **linker's** library search path.
- `-lschedule`: link `libschedule.a` (the linker adds the `lib` prefix and `.a` suffix).

Order matters. The linker processes inputs left to right, keeping a list of undefined symbols. When it reaches an archive, it extracts only members that resolve symbols already on that list. With the library *before* `main.o`, nothing is undefined yet, nothing is extracted, and the link fails:

```
$ gcc -Lbuild -lschedule build/obj/main.o -o schedgen
/usr/bin/ld: build/obj/main.o: in function `main':
main.c:82: undefined reference to `sched_config_default'
```

The linker map (`-Wl,-Map=…`) for the correct order shows which members were pulled in and why:

```
Archive member included to satisfy reference by file (symbol)
build/libschedule.a(diag.o)       build/obj/main.o (diag_init)
build/libschedule.a(loader.o)     build/obj/main.o (course_list_init)
build/libschedule.a(report.o)     build/obj/main.o (report_write_text)
build/libschedule.a(scheduler.o)  build/obj/main.o (sched_config_default)
build/libschedule.a(timeutil.o)   build/libschedule.a(loader.o) (tu_parse_time)
build/libschedule.a(common.o)     build/libschedule.a(loader.o) (sched_copy)
build/libschedule.a(csv.o)        build/libschedule.a(loader.o) (csv_open)
```

`ldd build/schedgen` lists only the C runtime (`libc.so.6`), with no run-time dependency on `libschedule`.

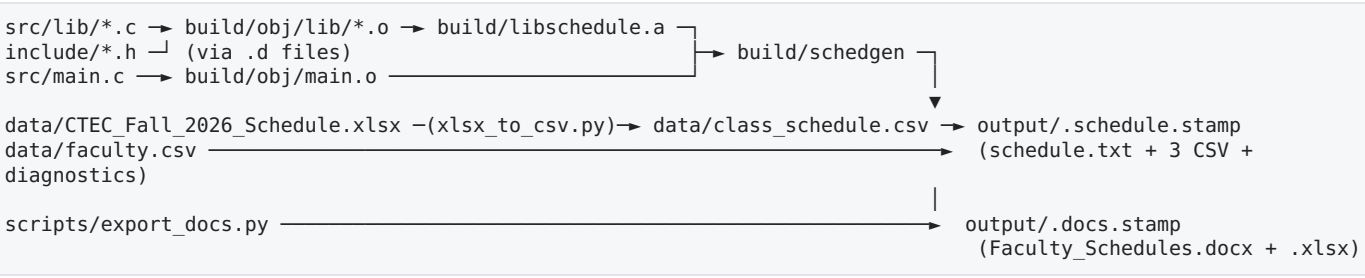
5. The Makefile

5.1 How make works

```
target: prerequisites
    recipe   # shell command(s), indented with a TAB
```

`make` builds a **dependency graph** from the rules and rebuilds a target only when it is missing or older than any prerequisite, so it does the minimum work after any change.

5.2 Dependency graph for this project



5.3 Techniques used

Technique	Where	Why
Pattern rule `$(OBJDIR)/lib/%.o: src/lib/%.c`	compile	One rule compiles every library source
Automatic variables `$@`, `$<`, `$^`	all rules	Recipes stay generic
`wildcard` / `patsubst`	`LIB_SRCS`, `LIB_OBJS`	A new `.c` in `src/lib/` needs no Makefile edit
`-MMD -MP` + `-include $(DEPS)`	compile	GCC writes `.d` files listing each object's headers, so a header edit recompiles exactly its users
Order-only prerequisites `| $(OBJDIR)`	compile	Directory must exist, but its timestamp must not trigger rebuilds
File-to-file conversion rule	`data/class_schedule.csv: data/…xlsx`	The CSV is regenerated only when the workbook changes
Stamp files	run, docs	One program run creates several files; the stamp represents the group
Exit-status handling in the recipe	run	Status 3 (data conflicts) continues; `STRICT=1` makes it fail
`.PHONY`, overridable variables	throughout	`make run SCHEDULE_XLSX=Spring.xlsx BUFFER=10` without editing the Makefile

5.4 Incremental builds (observed)

Change	What `make` did
Nothing	`make: Nothing to be done for 'all'.`
Edit `src/lib/timeutil.c`	Recompiled `timeutil.o` only → re-archived library → relinked program
Edit `include/model.h`	Recompiled only the files that include it (found via the `.d` files) → re-archived → relinked
Replace the `.xlsx`, then `make docs`	No C compilation; re-exported CSV → re-ran `schedgen` → re-ran the Python exporter

5.5 Python steps

`make docs` depends on the schedule stamp, which depends on the program, the exported CSV (which depends on the workbook) and the faculty file. One command therefore runs only the stages that are out of date, in order:

```
        python3 scripts/xlsx_to_csv.py data/CTEC_Fall_2026_Schedule.xlsx data/class_schedule.csv
        ./build/schedgen -o output -b 15 data/class_schedule.csv data/faculty.csv
        python3 scripts/export_docs.py --input output --docx output/Faculty_Schedules.docx --xlsx
output/Faculty_Schedules.xlsx
```

Both scripts use `openpyxl` / `python-docx` (`requirements.txt`; `make venv` installs them). The exported CSV is shipped, so the C build and `make run` work even on a machine without those packages.

6. Scheduling logic

6.1 Weekly grid and sessions

For each professor, the week is 7 days × 96 slots of 15 minutes, each FREE, CLASS, BUFFER or OFFICE:

1. Every scheduled meeting from **both** 7R1 and 7R2 is marked **CLASS**. One office-hours schedule is then valid for the entire semester, and students see the same hours all term.
2. `BUFFER` minutes (default 15) before and after each class are protected. This enforces *“Instructors are not to be interrupted during class times.”*
3. Asynchronous and TBA sections occupy no time.

6.2 Lecturer-supplied office hours

Entries in `fixed_office_hours` (e.g. `MW 3:00 PM-4:00 PM`) are processed first. Those overlapping a class are **dropped** with a warning. Those only inside the buffer are **kept** (the lecturer’s choice) with an info message. The CTEC faculty file ships with this column empty; the end-to-end test exercises it.

6.3 Generated office hours

Required weekly hours: `oh_hours_per_week`, or the policy default **max(3 hours, 1 hour per section)**. Preferred days default to the days the professor already teaches. This matters for a program where most sections meet in the evening and many instructors are on campus only two days a week. Each candidate block inside the availability window is scored:

Rule	Score	Rationale
Day on which the professor teaches	+30	Already on campus
Block touches a class buffer or an existing office-hours block	+20	“Office hours before class”; contiguous blocks
Each block already on that day	−50	Spreads hours across teaching days
Distance from that day’s classes	−1 per 10 min	Keeps hours next to class, not 6 hours earlier
Day not preferred	−100	Used only if preferred days are full

The best block is placed; ties go to the earliest day and time, so output is deterministic. Touching generated blocks merge into one row. Example: Anthony Adedoyin teaches MW 4:00–6:50 PM (both sessions) and 7:00–9:50 PM (7R1), so 3 hours are required. The result is **Monday 1:45–3:45 PM** and **Wednesday 2:45–3:45 PM**, each ending at the 15-minute buffer before his 4:00 PM class.

6.4 Conflict detection

Check	Severity
Instructor teaching two overlapping sections **in the same session**	ERROR
Two sections sharing a **physical** room (including either half of `201/112`) at overlapping times in the same session	ERROR
Instructor text not found in the faculty file	ERROR
Impossible time, invalid days, duplicate section ID	ERROR (section kept as TBA or skipped)
End time before start with AM end (data-entry error)	WARNING (corrected)
`courses_taught` disagrees with the schedule; lecturer hours overlap a class; requirement cannot be met	WARNING

7. Error handling and robustness

- Every library function returns a `sched_status`; allocation failures propagate to `main` (exit status 4).
- All string copies are bounded (`sched_copy`, `snprintf`); no `strcpy`/`sprintf`.
- Both input files are loaded before stopping, so a single run reports every input problem.
- Exit codes: `0` success, `1` usage, `2` input file unreadable, `3` data conflicts present, `4` output error.

8. Input file location: local vs. Internet

Decision: both input files are stored locally on the laboratory computer in the project’s `data/` directory. The C program opens them with standard C file I/O (`fopen`) using the two paths given on the command line.

Reasons:

1. **Reproducibility and grading.** The same inputs always produce the same output. A remote schedule is updated by the department during registration and could change between the student’s run and the instructor’s.

2. **No network dependency.** Lab machines may be offline, behind a proxy, or restrict outbound traffic.
3. **Simplicity and portability of the C code.** Standard C has no networking; downloading inside the program would require an external library such as libcurl, unrelated to the project's goals.
4. **Privacy.** The faculty file holds names and contact details; keeping it local avoids publishing it.
5. **Separation of concerns.** Where data comes from is independent of how it is processed; the program takes file paths, so any source works.

Internet option. If the department publishes the files online, the Makefile has an optional target that downloads them into `data/` with `curl`, after which the normal pipeline runs unchanged:

```
make fetch SCHEDULE_URL=https://.../schedule.xlsx FACULTY_URL=https://.../faculty.csv
make docs
```

Downloads go to a temporary file and are moved into place only after `curl --fail` succeeds, so a failed download never overwrites good local data. The fresh workbook is newer than the CSV export, so `make` regenerates everything downstream automatically.

9. Testing

Test	Result
`make test`: 104 unit checks (time/day parsing incl. Excel `4:00:00 PM`; CSV edge cases and header matching; loading the department format: ID split, session, async, AM→PM repair, TBA; alias matching; physical-room detection; session-aware conflicts; scheduler invariants: no office hours overlap any session's class + buffer, touching blocks merged, teaching-day default)	All pass
End-to-end: broken inputs in `tests/data/` must exit 3 and report instructor and split-room double-booking, unknown instructor, lecturer hours overlapping a class, AM→PM repair, and TBA; a same-slot section in the other session must **not** be flagged	Pass
End-to-end: one input file instead of two must exit 1	Pass
AddressSanitizer + UndefinedBehaviorSanitizer on the CTEC data and the unit tests	No findings
Valgrind `--leak-check=full` on the CTEC data	No leaks, no errors
GCC `-Wall -Wextra -Wpedantic -Wshadow -Wformat=2`; Clang with `-Wconversion`	Zero warnings
Ruff (Python, 120-character lines)	Clean
`make test-kb`: 35 Prolog rule tests (plunit): time formatting and overlap, name matching, who-teaches-what, office-hours labelling, availability, policy, and conflict rules (injected clashes must be found; 7R1 vs 7R2 must not clash)	All pass
`make test-kb`: 22 Python tests: fact-generator helpers (atom escaping, times, days, rooms, name tokens) and	All pass

question answering in six phrasings of “What classes does Dr Tankeh teach?” plus the full schedule in the sample’s layout, office hours, rooms, availability, misspelled names and raw queries	
End-to-end: CS input files → C program → `gen_prolog.py` → Prolog; the answer to “What classes does Dr Tankeh teach?” must list COSC 330, 369 and 631	Pass
Cross-check: the Prolog conflict rules find exactly the same 3 conflicts in the CTEC data as the C program	Pass

10. Sample output

From `output/schedule.txt`:

```
 Anthony Adedoyin (CT01)
 Department: Computer Technology (CTEC) | Office hours location: Office TBA
 Sections: 3 | Students enrolled: 53

CLASS SCHEDULE
+-------------------------------------------+---------+--------------------------------------+----------+
| Course                                    | Session | Times                                | Location |
+-------------------------------------------+---------+--------------------------------------+----------+
| CTEC 350.170 Prin & Meth of Intru Det & Pre | 7R1     | Monday & Wednesday 4:00 PM to 6:50 PM | 108      |
| CTEC 435.170 UNIX System Administration     | 7R1     | Monday & Wednesday 7:00 PM to 9:50 PM | 108      |
| CTEC 350.180 Prin & Meth of Intru Det & Pre | 7R2     | Monday & Wednesday 4:00 PM to 6:50 PM | 108      |
+-------------------------------------------+---------+--------------------------------------+----------+

OFFICE HOURS  (required 3.00 h/week, assigned 3.00 h/week; valid for both sessions)
+-----------+--------------------+------------+-------------+
| Day(s)    | Hours              | Location   | Arranged by |
+-----------+--------------------+------------+-------------+
| Monday    | 1:45 PM to 3:45 PM | Office TBA | Generated   |
| Wednesday | 2:45 PM to 3:45 PM | Office TBA | Generated   |
+-----------+--------------------+------------+-------------+
NOTE: Instructors are not to be interrupted during class times.
```

A section with repaired or missing data is marked with * and explained under the table:

```
| CTEC 120.170 Prin of SecureCoding Using Jav | 7R1     | Monday & Wednesday 12:00 PM to 3:15 PM * | 116      |
| CTEC 125.555 Intro to Python programming    | 7R1     | Online asynchronous (no set time)        | Online   |
  * CTEC 120.170: End time corrected from 3:15 AM to 3:15 PM
```

The Word document opens with a **Data Issues** page, followed by one page per professor in the sample’s table style. The Excel workbook has Summary, Classes, Office Hours and Data Issues sheets (errors highlighted), plus one sheet per professor. The text report also contains a weekly grid per professor showing which session each class belongs to (`7R1`, `7R2`, or `BOTH`).

11. Midterm: answering questions with Python and Prolog

11.1 Goal

The schedules produced in the second project are documents: a person has to read them to find an answer. The midterm adds a **question-answering** layer, so a student or the department can ask directly:

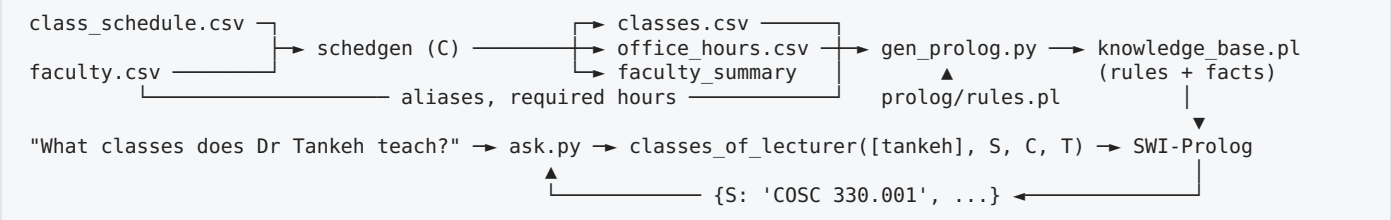
```
$ make ask Q="What classes does Dr Tankeh teach?"
```

Python and Prolog split the work the way the assignment describes:

Component	Role	File
C program (unchanged)	Reads the two input files, validates and normalises them, assigns office hours	`build/schedgen`
Python, step 1	**Reads** the schedule data and **generates the facts and rules** for Prolog	`scripts/gen_prolog.py`
Prolog knowledge base	**Stores knowledge**: part 1 rules (from the rule library `prolog/rules.pl`), part 2 facts	`output/knowledge_base.pl`
Python, step 2	Front end: recognises the question, builds a Prolog query, formats the answer	`scripts/ask.py`
SWI-Prolog (via `pyswip`)	**Interpreter**: answers each query by searching the facts with the rules	—

No answer is computed in Python. Python only translates: data → facts, and question → query → text.

11.2 Pipeline



Why Python reads the C program's output rather than the raw input files. The C program already resolves the three spellings of an instructor's name, repairs "3:15 AM" to "3:15 PM", classifies asynchronous and TBA sections and generates the office hours. Re-implementing that in Python would create a second set of rules that could disagree with the first. Reading the normalised CSV keeps one source of truth, and the knowledge base contains exactly the schedule that the Word and Excel documents show. The faculty file is read directly as well, for the name aliases used to match questions.

One knowledge base file. `gen_prolog.py` writes `output/knowledge_base.pl` in two parts: **part 1, rules**, copied from the rule library `prolog/rules.pl`, and **part 2, facts**, generated from the data. Python therefore produces everything Prolog needs, and Prolog loads it with a single `consult("knowledge_base.pl")`, as in the `pyswip` example in the assignment. The rules are kept in their own source file because they do not depend on the data: the same rules serve the CTEC schedule and the CS department, and they can be tested on their own.

11.3 Facts: what Prolog knows

`gen_prolog.py` writes one fact per piece of information. Every value is a **quoted atom** (`'CTEC 350.170'`, `'O\'Brien'`), so any text from the data is a valid Prolog term, and times are **minutes after**

midnight (`960` = 4:00 PM), so rules can compare them with `<`.

Fact	Meaning	Example
`faculty(Id, Name, Dept, Office, Email, Phone)`	A lecturer	`faculty('CS01', 'Dr. Tankeh', 'Computer Science (COSC)', 'CS 214', '', '')`
`alias(Id, Listed)`	A spelling used in the schedule	`alias('CS01', 'Tankeh, Dr.')`
`name_token(Id, Word)`	A lower-case name word (titles, initials dropped)	`name_token('CS01', tankeh)`
`course(Code, Title, Credits)`	A course	`course('COSC 369', 'Software Engineering I', 3)`
`section(Sec, Code, Session, Meeting, Enrolled)`	A section	`section('COSC 369.001', 'COSC 369', '', scheduled, 30)`
`teaches(Id, Sec)`	Lecturer teaches section	`teaches('CS01', 'COSC 369.001')`
`meets(Sec, Day, Start, End)`	One weekly meeting	`meets('COSC 369.001', tuesday, 570, 645)`
`room(Sec, Room)` / `room_listed(Sec, Text)`	Physical room(s) / room as listed	`room('CTEC 120.171', '112')`
`office_hours(Id, Day, Start, End, Location, Source)`	One office-hours block	`office_hours('CS01', wednesday, 900, 1020, 'CS 214', lecturer)`
`office_hours_required(Id, Minutes)`	Weekly requirement applied	`office_hours_required('CS01', 660)`
`data_issue(Severity, Message)`	A problem the C program reported	`data_issue(error, 'Room 112 is double-booked: ...')`

The CTEC schedule produces 602 facts. The file is regenerated by `make` whenever the C output or the faculty file changes, and contains no timestamp, so the same input always gives a byte-identical knowledge base.

11.4 Rules: how the facts relate

The rules (part 1 of the knowledge base, maintained in `prolog/rules.pl`) are written by hand and are independent of any particular department. Each answers one kind of question.

Rule	Question it answers
`lecturer_named(Words, Id)`	Which lecturer is “Dr Tankeh”? (every word must be one of the lecturer’s name words)
`classes_of_lecturer(Words, Sec, Code, Title)`	What classes does Dr Tankeh teach?
`lecturer_of(Code, Name)`	Who teaches COSC 369? (each lecturer once, however many sections)
`course_title_contains(Text, Code)`	Which course is “operating systems”?
`class_on(Day, Sec, Start, End)` / `class_in_room(Room, Sec)`	Which classes meet on Tuesday / in room 108?
`office_hours_on(Day, Id, Start, End, Loc)`	Which office hours are on Wednesday?
`office_hours_course(Id, Day, Start, Code)`	Which course does this office-hours block serve?
`teaching_at(Id, Day, Time, Sec)` / `in_office(Id, Day, Time)`	Is Dr Tankeh available on Monday at 10 AM?

`room_conflict/3`, `instructor_conflict/3`, `office_hours_clash/3`	Are there any conflicts?
`teaching_load(Id, Sections, Credits)`, `meets_office_hours_policy(Id)`	How many sections? Is the policy met?

Two rules show what Prolog adds. The first labels each office-hours block with the course whose class that day is nearest in time, which reproduces the layout of the department's sample, *"Wednesday: 10:45 AM – 1:00 PM (COSC 330) & 3:00 – 5:00 PM (COSC 631)"*, without that pairing being stored anywhere:

```
office_hours_course(Id, Day, Start, Code) :-
    office_hours(Id, Day, Start, End, _, _),
    findall(Gap-C, ( teaches(Id, Section), meets(Section, Day, CS, CE),
                     section(Section, C, _, _, _), gap(Start, End, CS, CE, Gap) ), Pairs),
    msort(Pairs, [Best-_|_]),
    member(Best-Code, Pairs).
```

The second states the double-booking check declaratively. A room is double-booked when two different sections in the same part of the term meet in it on the same day at overlapping times:

```
room_conflict(Room, A, B) :-
    room(A, Room), room(B, Room), A @< B, clash(A, B).

clash(A, B) :-
    same_term(A, B),
    once(( meets(A, Day, S1, E1), meets(B, Day, S2, E2), overlaps(S1, E1, S2, E2) )).
```

On the CTEC data these rules find exactly the three conflicts the C program reports (rooms 112 and 111, and Ruth Agada). Two independent implementations agreeing is strong evidence that both are correct and that the facts are a faithful copy of the schedule.

11.5 How a question is answered

Take *"What classes does Dr Tankeh teach?"*:

1. **Recognise the question.** `ask.py` normalises the text (lower case, no trailing `?`) and tries a list of regular expressions in order. `what classes does (?P<who>.+?) teach` matches, with `who = "dr tankeh"`.
2. **Extract the name.** Titles and filler words are dropped: `["tankeh"]`.
3. **Build the query** and hand it to Prolog through `pyswip`: `teaches(Id, Section), section(Section, Code, Term, _, _), course(Code, Title, _), meeting_text(Section, When), room_text(Section, Room)`, with `Id` first bound by `lecturer_named([tankeh], Id)`.
4. **Prolog searches.** `lecturer_named` finds `'CS01'` through `name_token('CS01', tankeh)`; `teaches` then yields three sections, and `meeting_text` builds "Monday & Wednesday 9:30 AM to 10:45 AM" from the `meets` facts.
5. **Format.** `pyswip` returns each solution as a Python dictionary; `ask.py` sorts the rows by session, day and time and prints them in the sample's Course / Times / Location layout.

The `pyswip` calls follow the pattern in the assignment notes:

```
        from pyswip import Prolog

        prolog = Prolog()
        prolog.consult("output/knowledge_base.pl")   # rules + facts, generated by gen_prolog.py
        for row in prolog.query("lecturer_named([tankeh], Id), teaches(Id, Section)"):
            print(row["Section"])               # COSC 330.001, COSC 369.001, COSC 631.001
```

Questions it understands include: a lecturer's full schedule (classes and office hours, in the sample's layout); what classes a lecturer teaches (optionally on one day); who teaches a course, by code or by title words; a lecturer's office hours; which office hours or classes fall on a day; which classes are in a room; where and when a course meets; whether a lecturer is available at a given time; how many sections a lecturer teaches; conflicts and data issues; and the list of lecturers. Any Prolog goal can also be typed directly, e.g. `?- lecturer_of('CTEC 350', Name).` A misspelled name gets a suggestion ("Did you mean: Dr. D. B. Stone?"), and a name that is not in the data gets a clear answer instead of an empty list.

11.6 Running it

```
        sudo apt-get install swi-prolog-nox          # SWI-Prolog 8.4.2+ (Windows/macOS: swi-prolog.org installer)
        python3 -m pip install -r requirements.txt   # python-docx, openpyxl, pyswip
        make kb                                      # C program (if needed) -> output/knowledge_base.pl
        make ask Q="What classes does Dr Tankeh teach?"
        make ask                                     # interactive: type questions, 'quit' to leave
        make demo                                    # data/questions.txt -> output/qa_demo.txt
        make test-kb                                 # Prolog + Python tests and the end-to-end question
        swipl output/knowledge_base.pl               # use Prolog directly
```

Python and SWI-Prolog must both be 64-bit. If `pyswip` cannot find the Prolog library, set `SWI_HOME_DIR` to the directory printed by `swipl --dump-runtime-variables | grep PLBASE`.

`make` treats the knowledge base like any other generated file: `output/knowledge_base.pl` depends on the C program's output, the faculty file, the rule library and `gen_prolog.py`, so editing either input file and running `make ask` re-runs the C program, regenerates the facts, and answers from the new data.

11.7 Sample session

The CTEC Fall 2026 schedule does not include Dr Tankeh, so the system answers that question honestly:

```
Q: What classes does Dr Tankeh teach?
No lecturer named 'Dr Tankeh' is in the knowledge base (faculty file and class schedule).
```

To demonstrate the question from the assignment, `tests/data/cs_class_schedule.csv` and `tests/data/cs_faculty.csv` describe a small Computer Science department. Dr Tankeh's three courses and lecturer-supplied office hours come from the sample in the project specification; they are **input data**, not code. Running the same unmodified pipeline on these two files gives (`sample_output/qa_demo_cs.txt`):

```
Q: What classes does Dr Tankeh teach?
Dr. Tankeh (CS01) teaches 3 section(s), 9 credit hours:

CLASS SCHEDULE
  Course                          Times                                  Location
  COSC 330 (Systems Programming I)   Monday & Wednesday 9:30 AM to 10:45 AM  CS 311
  COSC 369 (Software Engineering I)  Tuesday & Thursday 9:30 AM to 10:45 AM  CS 108
  COSC 631 (Database Management)     Wednesday 5:00 PM to 7:30 PM            CS 313

Q: What is Dr Tankeh's schedule?
Dr. Tankeh (CS01) | Computer Science (COSC) | Office: CS 214

CLASS SCHEDULE
  Course                          Times                                  Location
  COSC 330 (Systems Programming I)   Monday & Wednesday 9:30 AM to 10:45 AM  CS 311
  COSC 369 (Software Engineering I)  Tuesday & Thursday 9:30 AM to 10:45 AM  CS 108
  COSC 631 (Database Management)     Wednesday 5:00 PM to 7:30 PM            CS 313

OFFICE HOURS
  Monday:    10:45 AM – 1:00 PM (COSC 330)
  Tuesday:   10:45 AM – 1:00 PM (COSC 369)
  Wednesday: 10:45 AM – 1:00 PM (COSC 330)
             & 3:00 – 5:00 PM (COSC 631)
  Thursday:  10:45 AM – 1:00 PM (COSC 369)
  Friday:
NOTE: Instructors are not to be interrupted during class times.

Q: Is Dr Tankeh free on Monday at 10 AM?
No. Dr. Tankeh is teaching COSC 330.001 on Monday at 10:00 AM. Instructors are not to be interrupted during class
times.
```

The second answer reproduces the sample in the specification: the Course / Times / Location table with courses written as “COSC 631 (Database Management)”, every weekday listed under OFFICE HOURS (Friday empty), each block labelled with its course, the “3:00 – 5:00 PM” style, and the closing NOTE. The Session column appears only when the data has part-of-term sessions, as the CTEC schedule does. On the CTEC data (`sample_output/qa_demo.txt`):

```
Q: Who teaches CTEC 350?
CTEC 350 Prin & Meth of Intru Det & Pre is taught by:
  Anthony Adedoyin (CT01): CTEC 350.170 [7R1], CTEC 350.180 [7R2]

Q: Is Dr Stone available on Monday at 3 PM?
No. Dr. D. B. Stone is teaching CTEC 120.170 on Monday at 3:00 PM. Instructors are not to be interrupted
during class times.

Q: Are there any conflicts?
3 scheduling conflict(s):
  Room 112 is double-booked: CTEC 120.171 [7R1] ... vs CTEC 294.170 [7R1] ...
  Room 111 is double-booked: CTEC 345.180 [7R2] ... vs CTEC 450.180 [7R2] ...
  Ruth Agada is double-booked: CTEC 402.180 [7R2] ... vs CTEC 435.180 [7R2] ...
```

11.8 Design decisions

- **Facts are generated, rules are written.** Facts change every term; rules change only when the questions do. Keeping them in separate files means new data never touches the logic.
- **Integers for times.** Minutes after midnight make overlap, availability and nearest-class rules plain arithmetic comparisons, and formatting back to “4:00 PM” is a single rule, `clock/2`.
- **Same notion of a room as the C program.** `Online`, `Dept`, `TBA` and similar are not physical rooms in either implementation, so both agree on what counts as a conflict.
- **Fail cleanly.** Every fact predicate is declared `dynamic`, so a query against an empty or partial knowledge base returns “no” instead of raising an error.
- **The front end stays thin.** Recognising a question is pattern matching; anything that is a

relationship between facts is a Prolog rule, testable on its own with plunit.

12. Limitations and future work

- The greedy scheduler is optimal per step, not globally; a constraint solver could balance hours across faculty.
- One office-hours schedule covers both sessions. For an instructor who teaches only in 7R2, a per-session schedule would free up the first half of the term.
- Office locations and contact details are not in the department data and must be completed by hand.
- The AM→PM repair assumes an end time earlier than the start is a data-entry error. It is always reported, never silent.
- The question front end recognises a fixed set of sentence patterns (about 20). A question phrased in an unexpected way gets a list of examples rather than an answer; a natural-language parser (or an LLM that emits the Prolog query) would widen coverage while keeping Prolog as the source of truth.
- Availability answers use the weekly pattern and do not yet distinguish the 7R1 and 7R2 halves of the term.

Appendix A – Submission contents

Path	Contents
`Makefile`	Build, library, csv, run, docs, test, inspect, fetch, clean; midterm: kb, ask, demo, test-kb
`README.md`	Compilation and run instructions
`include/*.h`	8 headers (7 modules + umbrella `schedule.h`)
`src/main.c`, `src/lib/*.c`	Driver + 7 library modules
`build/libschedule.a`	Static library (rebuilt by `make`)
`data/CTEC_Fall_2026_Schedule.xlsx`	Department class schedule (source)
`data/class_schedule.csv`	Input file 1 (CSV export of the workbook)
`data/faculty.csv`	Input file 2
`scripts/xlsx_to_csv.py`, `scripts/export_docs.py`, `requirements.txt`	Python steps
`scripts/gen_prolog.py`	Midterm: schedule data → Prolog facts
`scripts/ask.py`	Midterm: question front end (pyswip → SWI-Prolog)
`prolog/rules.pl`	Midterm: Prolog rule library (part 1 of the generated knowledge base)
`data/questions.txt`	Midterm: questions answered by `make demo`
`tests/`	C unit tests, conflict test data, Prolog rule tests (`test_rules.pl`), Python tests (`test_kb.py`), CS department used for the Dr Tankeh example (`data/cs_*.csv`)
`sample_output/`	`schedule.txt`, CSV files, `diagnostics.txt`, `.docx`, `.xlsx`, `knowledge_base.pl`, `qa_demo.txt`, `qa_demo_cs.txt` from a reference run

`docs/report.pdf`, `docs/report.md`	This report and its source